

Final Group Work Report: RISC-V Pipelined Processor on PYNQ-Z2

Oscar Edgardo Navarro Banderas

Matr. Nr. 03782200

Beyrem Hadj Fredj

Matr. Nr. 03798143

Leon Adomaitis

Matr. Nr. 03789537

20.02.2026

1 Table of Contents

Contents

1	Table of Contents	1
2	Executive Summary	3
3	Design Overview	3
3.1	Project Goal and Verification Concept	3
3.2	System Architecture	3
3.3	Memory Layout Decision	5
3.4	Integration Deltas from the Lab Baseline	5
4	Module Description and Implementation	5
4.1	Pipeline Structure	5
4.2	Architectural Modifications for FPGA Deployment	5
4.3	Decode and ALU Control Path	6
4.4	Hazard and BRAM Timing Handling	6
4.5	Program Behavior and DONE Protocol	6
5	Testing and Debugging	6
5.1	Pre-Hardware Simulation	6
5.2	Cross-Simulator Discrepancy Notes	7
5.3	Hardware-in-the-Loop Verification Flow	7
5.4	On-Board Execution Profile (Captured Run)	7
5.5	Debugging Experience: What Failed and Why	8
5.6	On-Board Failure Case and Fix	8
6	FPGA Implementation Analysis	8
6.1	Hierarchical Resource Utilization	8
6.2	Critical Path Analysis	9
6.3	Timing Performance and Fmax	9
7	Technical Discussion	10
7.1	Tradeoff: Split Memories vs Unified Memory	10
7.2	Reset Strategy on Board	10
7.3	What We Would Improve Next	10
8	Conclusion	10
9	Contributions	10
10	Repository and Submission Artifacts	11
11	Citations and Appendix	11
11.1	Citations	11

11.2 AIGC Disclosure	11
11.3 Appendix A: Final Submission Checks	11

2 Executive Summary

This report presents the final implementation and verification of our pipelined RISC-V processor on the PYNQ-Z2 board. The main objective was to move from a simulation-only design to a hardware-in-the-loop workflow where the PS side (ARM + Python) acts as testbench for the PL side core.

Our final setup loads instruction/data memories from Python, starts the core through GPIO reset control, polls a done flag, and checks sorted output data against a software golden result. We used separate instruction and data BRAM blocks to reduce integration risk and simplify debugging under time constraints.

Functionally, the end-to-end flow works on board. Based on the latest implementation notes, this report now includes concrete utilization and timing numbers.

3 Design Overview

3.1 Project Goal and Verification Concept

The goal of this project is to validate a pipelined RISC-V design in real FPGA hardware, not only in behavioral simulation. In our final flow, verification responsibility is split as follows:

- PS side prepares memory, controls execution, and validates results.
- PL side executes the sorting program and writes completion status.

This setup mirrors a realistic hardware bring-up process: software orchestrates and observes, while hardware computes.

3.2 System Architecture

The implemented block design includes:

- Zynq processing system,
- AXI interconnect path,
- AXI BRAM controllers for instruction and data memories,
- AXI GPIO for reset control,
- custom pipelined RISC-V IP,
- dual-port BRAM generators.

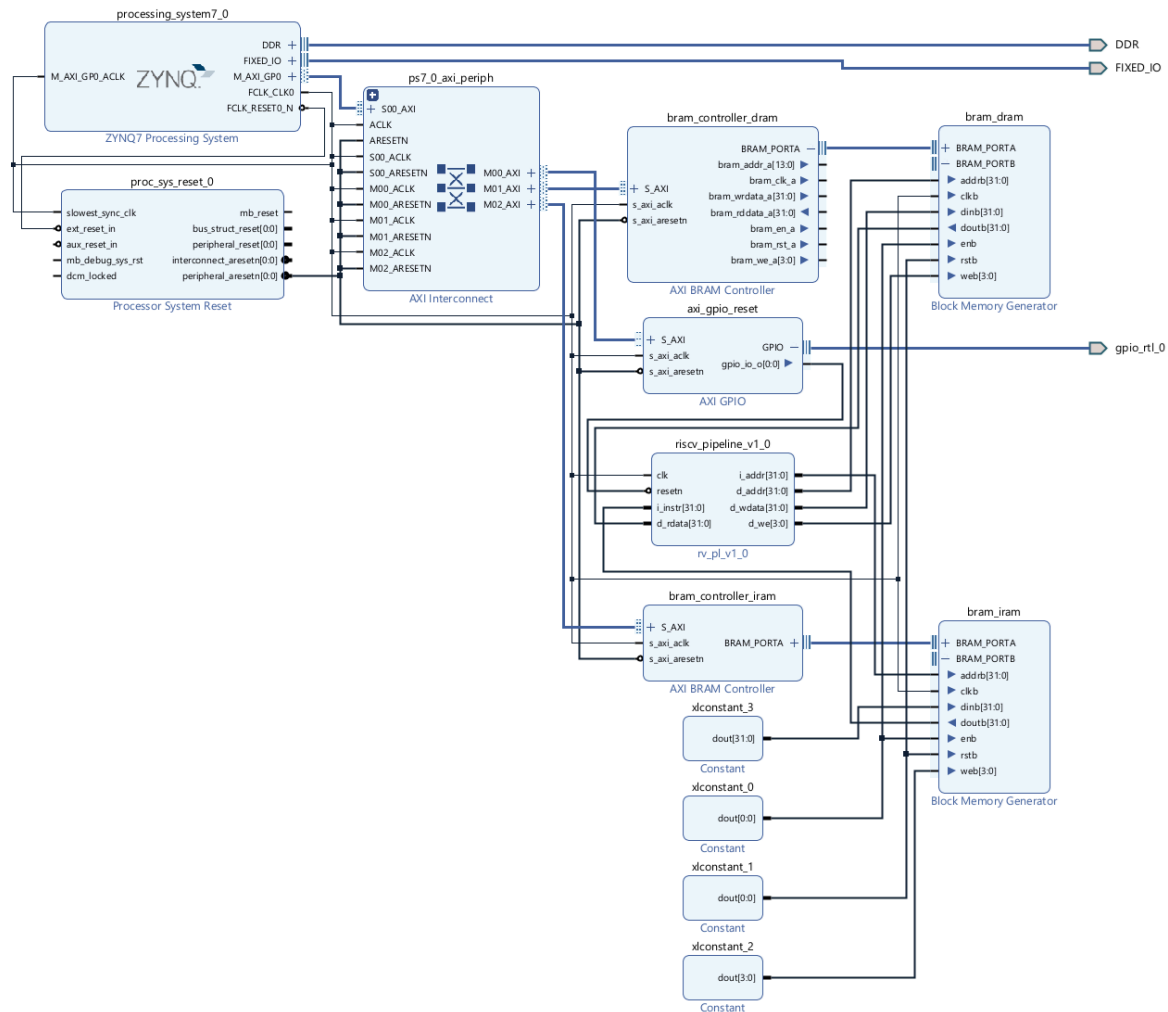


Figure 1: Vivado block design used for PS-PL integration and verification flow.

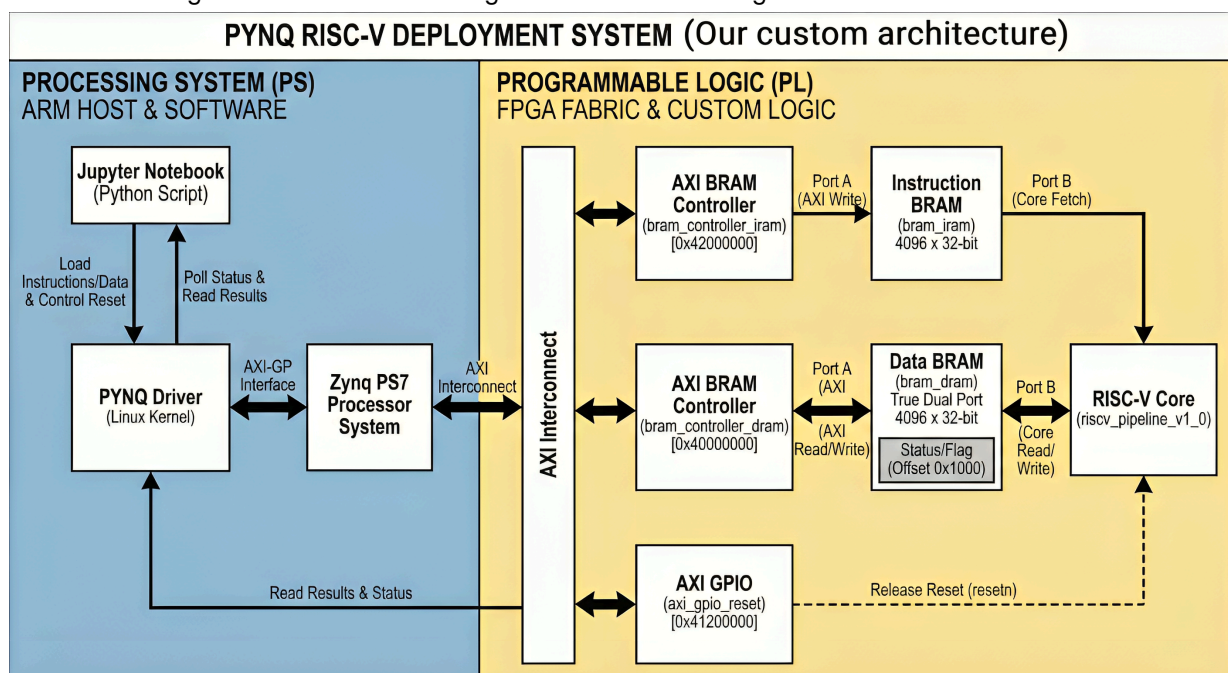


Figure 2: Custom PS-PL deployment architecture used during project integration and demo preparation.

During runtime, the Python notebook accesses three memory-mapped regions:

Block	Base Address	Role in verification
bram_controller_dram	0x40000000	Data memory writes/reads and DONE polling
bram_controller_iram	0x42000000	Instruction injection and readback verification
axi_gpio_reset	0x41200000	Core halt/run control

3.3 Memory Layout Decision

The assignment allows both unified and split memory approaches. We chose the split design (instruction BRAM + data BRAM). This was a practical decision: it reduces structural contention complexity and shortens debug cycles near deadline. A unified memory can be valuable as an optimization exercise, but it requires stricter arbitration and deeper stall policy validation.

3.4 Integration Deltas from the Lab Baseline

We started from the Lab 12 style PS-PL architecture and kept the proven communication structure, then introduced only the changes needed for the final project.

- Interconnect adaptation: AXI SmartConnect was replaced by AXI Interconnect in the final block design flow used by Vivado automation.
 - Memory split: a second AXI BRAM controller was added to support separate IRAM and DRAM blocks.
 - Custom core packaging: the `rv_pl` CPU wrapper was integrated as a reusable IP in the block design.
- This incremental strategy helped avoid unnecessary integration churn late in the project and made failures easier to localize.

4 Module Description and Implementation

4.1 Pipeline Structure

The processor uses five classic stages (IF, ID, EX, MEM, WB) and dedicated pipeline registers (`fd_register`, `de_register`, `em_register`, `mw_register`).

Stage	Main function	Representative modules
IF	PC update and instruction fetch	<code>MyPipelinedProcessor.v</code> , <code>MyProgramCounter.v</code> , <code>fd_register.v</code>
ID	Decode, immediate generation, register read	<code>MyController.v</code> , <code>MyControlUnit.v</code> , <code>MyInstructionDecoder.v</code> , <code>MyRegisterFile.v</code> , <code>de_register.v</code>
EX	ALU operations, branch target, operand forwarding	<code>MyALUDecoder.v</code> , <code>MyAlu.v</code> , <code>3mux.v</code> , <code>de_register.v</code> , <code>em_register.v</code>
MEM	Data memory access and write-enable control	<code>MyPipelinedProcessor.v</code> , <code>em_register.v</code> , <code>mw_register.v</code>
WB	Result selection and register write-back	<code>MyPipelinedProcessor.v</code> , <code>mw_register.v</code>

4.2 Architectural Modifications for FPGA Deployment

Moving the lab core to FPGA required changes beyond straightforward synthesis:

1. Hazard unit extension for synchronous memory behavior: load and store interactions were re-checked under one-cycle BRAM latency assumptions, with explicit stall/forward coordination.
2. Top-level BRAM interface exposure: `rv_pl` exports instruction and data memory ports directly (`i_addr/i_instr`, `d_addr/d_wdata/d_we/d_rdata`) for clean block-design integration.
3. Instruction buffering during stalls: an explicit buffer in fetch/decode path prevents instruction corruption across stall-release transitions.

These modifications were central to board stability and cannot be inferred from functional simulation alone. At coding style level, sequential state updates were constrained to clocked `always @(posedge clk)` processes so BRAM-latency assumptions remain explicit. This prevented mixed combinational/sequential behavior from hiding timing bugs during bring-up.

4.3 Decode and ALU Control Path

Control logic is split into two levels:

- `MyControlUnit.v` decides high-level instruction class behavior (`reg_write`, `mem_write`, `alu_src`, `result_src`, `branch`, `jump`, `alu_op`).
- `MyALUDecoder.v` maps opcode/function bits to concrete ALU operations.

This split was useful in debugging because opcode-level issues and ALU-operation issues could be diagnosed independently.

4.4 Hazard and BRAM Timing Handling

The most important board-specific challenge is synchronous BRAM behavior. Read data is tied to clocked address phases, so pipeline control must maintain alignment across cycles.

The implemented hazard/timing strategy combines:

- RAW forwarding from M/W stages into E stage,
- load-use stall detection,
- BRAM latency stall gating,
- fetch PC registration for address/data alignment,
- instruction buffering during stall transitions,
- additional branch-related flush protection for stale fetch windows.

Without these mechanisms, a design may pass simplified simulation but still fail intermittently on board.

4.5 Program Behavior and DONE Protocol

The sorting workload processes 32 signed integers in place in DRAM (0x00 to 0x7C). Completion is signaled by storing 0xDEADBEAF at byte address 0x100.

During development, we used two program styles:

- an unrolled adjacent compare-swap generator (large instruction footprint, useful for deterministic traceability), and
- a compact loop-based bubble sort program (27 instructions) for on-board verification and faster debug iteration.

The final captured board run in the notebook/PDF uses the compact 27-instruction variant and verifies correct ordering of all 32 elements.

The constant build sequence is:

- `lui x12, 0xDEADC`
- `addi x12, x12, -337`
- `sw x12, 256(x0)`

This detail is important because it is both an ISA-constrained implementation choice and the synchronization contract with the host script.

5 Testing and Debugging

5.1 Pre-Hardware Simulation

The project includes a dedicated testbench (`tb_rv_pl.v`) with synchronous BRAM modeling (address registered on clock edge, data observed on the following cycle), which is closer to real FPGA behavior than idealized combinational memories.

The test sequence is intentionally progressive:

1. basic store operations,
2. store-then-load behavior,
3. load-use hazard case,

4. consecutive loads plus dependent arithmetic,
5. branch-taken behavior,
6. mini bubble sort (4 elements),
7. full 32-element sort with done-flag check.

This structure made it possible to isolate control and memory-timing issues before running the full workload. In team simulation notes, the full 32-element test completes in roughly 9,609 cycles, which matches the expected cost of bubble sort under a simple in-order pipeline with frequent memory traffic.

5.2 Cross-Simulator Discrepancy Notes

One important development lesson was simulator behavior mismatch. In team testing, all seven tests passed in iverilog, while selected tests failed in Vivado xsim due to unknown (x) propagation. The root causes were initialization assumptions:

- register file and selected control/data registers required explicit reset/initialization discipline, and
- BRAM-address staging registers in testbench needed deterministic startup values.

Treating initialization explicitly improved portability and reduced “works here, fails there” ambiguity.

Representative initialization pattern used during cleanup:

```
initial begin
    i_addr_reg = 32'b0;
    d_addr_reg = 32'b0;
end
```

5.3 Hardware-in-the-Loop Verification Flow

The Jupyter notebook (verify_submission.ipynb) follows a deterministic run sequence:

1. load overlay,
2. map MMIO peripherals,
3. execute memory and GPIO sanity checks,
4. inject program and input data,
5. release reset,
6. poll status flag,
7. compare output with Python sorted() golden result.

The captured notebook outputs show successful completion flag detection and correct ordering for all 32 result elements.

Before running the sorter, the notebook executes interface sanity checks for IRAM, DRAM, and GPIO. Typical checks include write/readback of patterns such as 0xDEADBEEF, 0xCAFEBAFE, and address-tagged values (0xAA000000 | offset) across offsets like 0x0, 0x4, 0x10, 0x7C, and 0x100. This step is important because it separates integration faults (addressing, MMIO mapping, reset control) from actual CPU-logic bugs.

5.4 On-Board Execution Profile (Captured Run)

Metric	Observed value
Program length	27 instructions (loop-based bubble sort)
Input size	32 signed integers
DONE flag address	0x100 (byte offset in DRAM)
DONE flag value	0xDEADBEAF
Observed completion time	approximately 0.0017 s
Result check	32/32 elements matched software golden output

This profile is useful in presentation and report defense because it ties together memory map, control flow, and verification outcome in one reproducible record.

5.5 Debugging Experience: What Failed and Why

The hardest bugs were not arithmetic errors; they were alignment and control problems created by synchronous memory timing and staged pipeline state.

Observed symptom	Technical reason	Fix applied
Unstable or inconsistent board output	Address/data timing mismatch around BRAM reads	Introduced BRAM-aware stalling and fetch alignment logic
Wrong instruction after stall release	Instruction stream advanced while valid fetch data lagged	Added instruction buffering for stall transitions
Control transfer irregularities	One-cycle stale fetch behavior after branch redirection	Added extra flush handling near branch redirect

A second lesson was tool behavior: RTL accepted in one flow can still expose stricter elaboration issues in another simulator. Cross-tool checks should start earlier in future iterations.

5.6 On-Board Failure Case and Fix

A representative board issue was instruction-sequencing corruption around stall windows: after a stall, the next decoded instruction could be misaligned with the intended fetch stream. The practical fix was to latch the instruction when entering stall and replay it safely until normal flow resumed.

```
always @(posedge clk) begin
    if (!resetsn || FD_flush_final)
        instr_buf_valid <= 1'b0;
    else if (!F_stall)
        instr_buf_valid <= 1'b0;
    else if (F_stall && !instr_buf_valid) begin
        instr_buf      <= i_instr;
        instr_buf_valid <= 1'b1;
    end
end
end
wire [31:0] fd_instr_in = instr_buf_valid ? instr_buf : i_instr;
```

This fix is a good example of why board timing behavior must be treated as a first-class design constraint, not a post-processing detail.

6 FPGA Implementation Analysis

This section summarizes the post-implementation data provided from Vivado and explains the observed bottlenecks.

6.1 Hierarchical Resource Utilization

Module	LUTs	FFs	LUT-Logic	LUT-Mem	BRAM
riscv_pynq_wrapper	2528	2460	2406	122	8
riscv_pipeline_v1_0 (total)	976	596	932	44	0
Branch_Adder	0	0	0	0	0
HazardUnit	0	1	0	0	0
ProgramCounter	0	32	0	0	0
PC_Adder	0	0	0	0	0
PC_R_Adder	1	0	1	0	0
FD_register	136	94	136	0	0
DE_Register	438	183	438	0	0
EM_Register	10	116	10	0	0

MW_Register	337	104	337	0	0
ALU	16	0	16	0	0
RegisterFile	44	0	0	44	0

The dominant LUT consumer inside the core is DE_Register. This matches expectations: the decode-to-execute boundary carries a wide bundle of operand buses, register indices for hazard decisions, and multiple control signals. The logic cost is therefore concentrated at this stage boundary rather than in a single arithmetic block.

6.2 Critical Path Analysis

From the implementation notes, the reported worst path starts near a memory-stage register index signal and ends at an execute-stage immediate register (.../M_rf_a3_reg[4]/C -> .../PLR2/E_ext_reg[21]/R).

This observation suggests that the timing limit is not driven primarily by BRAM setup nor by one ALU operator. Instead, the bottleneck is more consistent with combined control propagation and selection logic around execute-stage forwarding/selection paths.

Name	Slack	Levels	High Fanout	From	To
Path 1	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_ext_reg[21]/R
Path 2	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rf_rd1_reg[12]/R
Path 3	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rf_rd2_reg[6]/R
Path 4	-2.399	14	183	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR3/M_rf_a3_reg[4]/C	riscv_pynq_i/riscv_pipeline_v1_0/inst/PLR2/E_rs1_reg[3]/R

Figure 3: Vivado timing report excerpt for the top violating paths.

In the captured timing table, multiple top paths share the same characteristics: Slack = -2.399 ns, Levels = 14, and High Fanout = 183, with a common source around PLR3/M_rf_a3_reg[4]/C and endpoints in PLR2 registers (E_ext, E_rf_rd1, E_rf_rd2, E_rs1). This pattern supports the interpretation that control/metadata fanout around the decode-execute boundary is a major timing contributor.

In practice, this means delay is distributed across control/selection depth:

- forwarding mux selection,
- branch-target/operand path interaction,
- execute-stage metadata propagation.

Representative execute-stage structure:

```
ThreeMux MuxA (
    .in0(E_rf_rd1), .in1(W_result), .in2(M_alu_o),
    .sel(ForwardAE), .out(E_src_a_forwarded)
);

ThreeMux MuxB (
    .in0(E_rf_rd2), .in1(W_result), .in2(M_alu_o),
    .sel(ForwardBE), .out(E_src_b_forwarded)
);

Adder Branch_Adder (
    .a(E_pc), .b(E_ext), .sum(E_target_pc)
);

assign E_alu_src_b = (E_sel_alu_src_b) ? E_ext : E_src_b_forwarded;
```

6.3 Timing Performance and Fmax

Using the reported values:

- WNS = -2.399 ns

- $T_{\text{constraint}} = 10.000 \text{ ns}$ (100 MHz target)

A negative setup slack means the design misses the target period and requires a longer effective clock period:

- $T_{\text{clk,min}} = T_{\text{constraint}} + |WNS| = 12.399 \text{ ns}$
- $F_{\text{max}} = 1 / T_{\text{clk,min}} = 80.65 \text{ MHz}$ (approx.)

These values explain why timing closure at 100 MHz was not achieved in the cited run, even though functional verification passed.

7 Technical Discussion

7.1 Tradeoff: Split Memories vs Unified Memory

Our split-memory design emphasizes implementation stability and verification clarity. A unified BRAM approach can reduce block count but pushes more complexity into pipeline arbitration and fetch/data conflict handling.

Given the deadline, the split design provided a better risk profile and a clearer path to a reproducible demo.

7.2 Reset Strategy on Board

Board reset handling is critical for repeatability. We control reset through AXI GPIO because this is deterministic, scriptable, and easy to integrate with automated tests. A physical button remains useful for demonstration, but must include debounce treatment to avoid jitter-related false triggers.

7.3 What We Would Improve Next

If we continue after the final submission, the next steps would be:

- automate Vivado report extraction into report tables,
- add cycle/performance counters for deeper profiling,
- optionally add a unified-memory variant for direct architectural comparison.

8 Conclusion

The project met its objective: the pipelined RISC-V core was integrated on PYNQ-Z2 and validated through a reproducible PS-driven hardware-in-the-loop workflow. Sorting results were written correctly to memory, and the done-flag protocol enabled reliable completion detection from the host side. The key engineering takeaway is that FPGA correctness depends on timing-aware control decisions at least as much as on instruction-level functionality. BRAM timing, hazard policy, and reset orchestration are central to making the system stable in real hardware.

From a course perspective, the most relevant lesson is the transition from “functionally correct RTL” to “implementation-aware digital design.” The final system only became reliable after we treated memory timing and pipeline control as one coordinated problem.

If we were to iterate once more, our first optimization target would be timing closure around execute/control fanout paths (retiming or deeper control-path partitioning), followed by automated extraction of implementation metrics into the report workflow.

9 Contributions

The workload was split fairly based on available time and task type.

Team Member	Primary Contributions	Weight
-------------	-----------------------	--------

Oscar Banderas	Main RTL implementation, pipeline/hazard refinements, Vivado integration support	High
Beyrem Hadj Fredj	Implementation support, verification flow support, on-board validation/debugging	High
Leon Adomaitis	Report writing, technical documentation, result consolidation, submission preparation	Medium

Shared contribution: all members participated in debugging decisions and final integration discussions.

10 Repository and Submission Artifacts

Project repository used for submission:

- <https://github.com/Oscar27NX/hdl-ws25-26-final-project.git> (branch: RV_PYNQ)

Main artifact groups available in the repository:

- RTL source and pipeline modules,
- block-design wrapper and integration files,
- simulation testbench and hex programs,
- bitstream and hardware handoff (.bit, .hwh),
- verification notebook and software-side test material.

11 Citations and Appendix

11.1 Citations

- Final project requirements PDF (course handout, January 27, 2026).
- PYNQ documentation for overlay/MMIO workflow: <https://pynq.readthedocs.io/>.
- Repository artifacts used as primary evidence: RTL sources, simulation testbench, verification notebook, block-design wrapper, and hardware handoff file.
- TUM Typst theme reference: <https://github.com/lufixSch/tum-templates-typst>.

11.2 AIGC Disclosure

AI assistance was used only for language-level style refinement. Technical content, implementation claims, and verification statements were reviewed against repository artifacts by the team.

11.3 Appendix A: Final Submission Checks

1. Re-check that utilization and timing numbers correspond to the exact final implementation run.
2. Optionally add additional waveform/schematic screenshots to strengthen debugging evidence in presentation.